Institut Supérieur d'Informatique et de Mathématiques de Monastir

Année Universitaire 2025-2026

Devoir Surveillé

Classe	Matière	Enseignant
L3 – INFO	Développement Mobile	Dr. Naoufel KHAYATI

Date	Vendredi 21 novembre 2025	**Nombre de Pages**	5
Durée	1H00 (10h00 – 11h00)	**Documents**	Non Autorisés
Barème	Ex1 : 4 pts ; Ex2 : 6 pts ; Ex3 : 10 pts	**Durée estimée / exercice**	Ex1 : 10 min ; Ex2 : 20 min ; Ex3 : 30 min

Exercice 1 (*4 pts*)

Cochez la / les bonne(s) réponse(s).

<u>NB : boutons radio = réponse unique / cases à cocher = réponses multiples</u>

1. Quel est le fichier de configuration servant à déclarer toutes les activités d'une application Android ?
   ❍ *AndroidManifest.xml* ❍ *Activities.json* ❍ *MainActivity.kt* ❍ *build.gradle*

2. Pour une application Android dont la langue par défaut est l'anglais, où doit être placé le fichier *strings.xml* de la langue allemande ?
   ❍ *res/strings-de* ❍ *res/values-de* ❍ *res/values-ger* ❍ *res/values*

3. Dans un projet Android, dans quel dossier peut-on trouver des images classiques (PNG, JPG, GIF, etc.) et des images vectorielles ?
   ❍ *res/images* ❍ *res/mipmap* ❍ *res/drawable* ❍ *res/layout*

4. Soit la déclaration suivante : `var obj : type? = null`
   Quelle instruction servira à appeler la fonction *f()* sur l'objet *obj* sans provoquer *NullPointerException* ?
   ❍ *obj!.f()* ❍ *obj!!.f()* ❍ *obj.f()* ❍ *obj?.f()*

5. L'activation du *ViewBinding* se fait dans le fichier :
   ❍ *settings.gradle* ❍ *build.gradle* ❍ *AndoidManifest.xml* ❍ *viewbinding.xml*

6. Dans quel répertoire du projet Android doit-on stocker les images afin qu'elles s'affichent correctement sur différentes densités d'écrans ?
   ❍ *res/images* ❍ *res/mipmap* ❍ *res/pictures* ❍ *res/assets*

7. Comment s'appelle la fonction qu'on trouve par défaut dans toute activité Android ?
   ❍ *onStart()* ❍ *onRun()* ❍ *onCreate()* ❍ *main()*

8. Le langage phare utilisé pour développer des applications iOS est :
   ❍ *Java* ❍ *Kotlin* ❍ *C++* ❍ *Swift*

9. Dans une application Android, pour pouvoir gérer le clic sur un bouton, il faut associer à ce dernier un listener à travers la méthode :
   ❍ *setOnClickListener()* ❍ *setOnClick()* ❍ *setListener()* ❍ *onClick()*

10. Pour publier une application dans le Play Store et l'App Store en utilisant une seule base de code, quel type de développement est le plus adapté ?
    ❍ *Natif* ❍ *Hybride* ❍ *Low-code* ❍ *Sans code*

11. Gradle est un : ***(Choisir 2)***
    ☐ *Gestionnaire de dépendances* ☐ *Compilateur* ☐ *Outil de build* ☐ *Emulateur Android*

12. Parmi les éléments suivants, lesquels NE SONT PAS des ressources ? ***(Choisir 2)***
    ☐ *les couleurs* ☐ *les layouts* ☐ *les plugins* ☐ *les intents*

13. Parmi les éléments de cette liste, les noms d'OS mobiles sont : ***(Choisir 2)***
    ☐ *Ionic* ☐ *Tizen* ☐ *Symbian* ☐ *Flutter*

14. Quelle combinaison de qualificateurs est valide pour un dossier de ressources Android ?
    - ❍ *drawable-en-land-mdpi*
    - ❍ *layout-1280x720-fr*
    - ❍ *mipmap-port-night-France*
    - ❍ *values-v34-land-hdpi*

15. Quel est l'avantage principal du développement mobile natif ?
    - ❍ Un seul code pour toutes les plateformes.
    - ❍ De meilleures performances et un accès complet aux API du système
    - ❍ Déploiement uniquement sur Android
    - ❍ Dépendance totale du navigateur

16. Pour les applications Android à base de JetPack Compose :
    - ❍ Seul le langage Java est supporté.
    - ❍ Seul le langage Kotlin est supporté.
    - ❍ Seul le langage Dart est supporté.
    - ❍ Java et Kotlin sont les deux langages supportés

Exercice 2 (*6 pts*)

1. Citez trois plateformes croisées (*cross-platforms*) pour le développement mobile **(0,75 pt)**

   a) …… b) …… c) ……

   Flutter / Ionic / React Native / .NET MAUI / Kotlin Multiplatform ...

2. Citez trois noms de versions Android venues après Android 8 (*Oreo*) **(0,75 pt)**

   a) …… b) …… c) ……

   Pie / Queen Cake / Red Velvet Cake / Snow Cone / Tiramisu / Upside Down Cake / Vanilla Ice Cream / Baklava

3. Citez trois qualificateurs à utiliser avec les noms des dossiers de ressources **(0,75 pt)**

   a) …… b) …… c) ……

   Langue (et Région) / Taille de l'écran / Orientation de l'écran / Résolution de l'écran / Version de l'API Android…

4. Citez les noms de trois classes de ressources qu'on peut trouver dans R **(0,75 pt)**

   a) …… b) …… c) ……

   color / string / dimen / mipamp / drawable / layout ...

5. Citez trois objets d'interface couramment utilisés dans une interface Android (*Donnez leurs noms tels qu'ils sont définis par Android*). **(0,75 pt)**

a) ..... b) ..... c) .....

EditText / TextView / Button /

6. Citez trois propriétés courantes d'un TextView configurables dans le code XML. **(0,75 pt)**

   a) ..... b) ..... c) .....

   id / text / textColor / textSize / layout_width / layout_height

7. Citez trois outils nécessaires au développement Android. **(0,75 pt)**

   a) ..... b) ..... c) .....

   Langage de Programmation (Kotlin ou Java) / JRE (Java Runtime Environment) / IDE (Android Studio) / Android SDK (Software Development Kit)

8. Citez trois éléments définis / déclarés dans le fichier AndroidManifest.xml. **(0,75 pt)**

   a) ..... b) ..... c) .....

   Nom de l'application / Package de l'application / Icône de l'application / Activités / Permissions ...

Exercice 3 (*10 pts*)

Soit une application Android à base de vues XML.

A noter que le *ViewBinding* est déjà activé et que l'objet associé utilisé dans *MainActivity* est noté « binding ».

1. Complétez les trois éléments manquants dans le code Kotlin ci-dessous qui se trouve dans le fichier *MainActivity.kt*. **(0,75 pt)**

```
override fun onCreate(savedInstanceState : Bundle?) {
        super.onCreate(savedInstanceState);
        setContentView(binding.root);
```

2. On souhaite ajouter dans l'interface de cette activité un champ de saisie permettant à l'utilisateur d'entrer son nom.
   On vous fournit le code XML **incomplet** suivant :

```
<EditText
    android:id="..."
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="..."
    android:textColor="..."
    android:textSize="..." />
```

a. Donnez la ligne qui attribut l'identifiant *edtUsername* à cet EditText. **(1 pt)**

```
android:id="@+id/edtUsername"
```

b. Le texte d'indication (hint) doit être lu depuis les ressources, à partir de la chaîne nommée "hint_user".

Donnez la ligne qui attribut cette ressource à l'EditText. **(1 pt)**

```
android:hint="@string/hint_user"
```

c. On souhaite que le texte saisi dans cet EditText soit d'une couleur et d'une dimension données.

i. Faites le nécessaire pour créer ces deux ressources ayant des noms et des valeurs de votre choix. **(2 pts)**

```
<color name="mycolor">#00FFAA</color>

<dimen name="mydimen">20sp</dimen>
```

ii. Appliquez ces deux ressources à l'EditText au niveau des propriétés adéquates. **(2 pts)**

```
android:textColor="@color/mycolor"

android:textSize="@dimen/mydimen"
```

3. Donnez le code permettant d'afficher un Toast saluant la personne dont le nom a été saisi dans l'EditText.
Par exemple, si on saisit « *Mohamed* », le message affiché serait « *Bonjour Mohamed* ».
(1,75 pt)

```
Toast.makeText(this, "Bonjour ${binding.edtUsername.text)",
Toast.LENGTH_LONG).show()
```

4. On souhaite maintenant que l'application navigue vers une activité nommée *ProfileActivity*.
Donnez le code nécessaire afin de répondre à ce besoin. **(1,5 pt)**

```
val intent = Intent(this, ProfileActivity::class.java)
startActivity(intent)
```

Bon Travail